

BLOC 2/3 POO :

Chapitre 1 :

```
package ch1;
```

```
public class Etudiant {
```

```
    private String nom;  
    private int age;  
    private int majorite = 18;
```

```
    public Etudiant(String nom) {  
        this.nom = nom;  
    }
```

```
    public Etudiant(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }
```

```
    public String getNom() {  
        return this.nom;  
    }
```

```
    public void setNom(String nom) {  
        this.nom = nom;  
    }
```

```
    public boolean memeNom(String unNom) {  
        boolean meme=false;  
        if (this.nom.equals(unNom)) {  
            meme = true;  
        }  
        return meme;  
    }
```

```
    public boolean memeNom2(Etudiant e) {  
        boolean meme=false;  
        if (this.nom.equals(e.nom)){  
            meme = true;  
        }  
        return meme;  
    }
```

```
    public boolean memeNom3(Etudiant nomEtu) {  
        boolean meme=false;  
        if (this.nom.equals(nomEtu.nom)){  
            meme = true;  
        }  
    }
```

```

        return meme;
    }

    public int getMajorite()
    {return this.majorite;
    }

    public boolean paramPasUtilmemeNom(Etudiant nomEtudiant) {
        boolean meme=false;
        if (this.nom.equals(nomEtudiant.nom)){
            meme = true;
        }

        return meme;
    }

    public String uneMemeChaine() {
        String chaine=" ";
        if (this.nom.equals("truc"))
        {chaine = "truc";
        }
        else
            if (this.nom.equals("truc2")) {
                chaine = "truc";
            } else {
                if (this.nom.equals("truc3")) {
                    chaine = "truc";
                }
            }
        return chaine;
    }
    public String unAgeOuAutre() {
        String chaine = "pastruc282148";
        if ((this.age == 2)|| (this.age == 8)
        || (this.age == 21) || (this.age == 48))
        {
            chaine = "truc282148";
        }

        return chaine;
    }
}

```

```

package ch1;
public class Etudiant2 {

```

```

    private String nom;
    private int age;
    private String etuPrenom;
    private int classement1;

    private int classement2;

```

```
private int niveau;
```

```
private int majorite = 18;
```

```
public Etudiant2(String nom) {  
    this.nom = nom;  
}
```

```
public Etudiant2(String nom, int age) {  
    this.nom = nom;  
    this.age = age;  
}
```

```
public String getNom() {  
    return this.nom;  
}
```

```
public void setNom(String nom) {  
    this.nom = nom;  
}
```

```
public String getEtuPrenom() {  
    return this.etuPrenom;  
}
```

```
public void setEtuPrenom(String etuPrenom) {  
    this.etuPrenom = etuPrenom;  
}
```

```
public int getclassement1() {  
    return this.classement1;  
}
```

```
public void setclassement1(int moyenne) {  
    this.classement1 = moyenne;  
}
```

```
public int getclassement2() {  
    return this.classement2;  
}
```

```
public void setclassement2(int classement2) {  
    this.classement2 = classement2;  
}
```

```
public int getNiveau() {  
    return this.niveau;  
}
```

```
public void setNiveau(int niveau) {  
    this.niveau = niveau;  
}
```

```
public boolean memeNom(String unNom) {  
    boolean meme=false;  
    if (this.nom.equals(unNom)) {  
        meme = true;  
    }  
    return meme;  
}
```

```

public boolean memeNom1(Etudiant etu) {
    boolean meme=false;
    if (this.nom.equals(etu.getNom())) {
        meme = true;
    }
    return meme;
}

    public boolean memeNom2(Etudiant e) {
        boolean meme=false;
        if (this.nom.equals(e.getNom())) {
            meme = true;
        }
        return meme;
    }

public boolean memeNom3(Etudiant nomEtu) {
    boolean meme=false;
    if (this.nom.equals(nomEtu.getNom())) {
        meme = true;
    }
    return meme;
}

public int getMajorite() {
    return majorite;
}

public boolean paramPasUtilmemeNom(Etudiant nomEtud) {
    boolean meme=false;
    if (this.nom.equals(nomEtud.getNom())) {
        meme = true;
    }
    return meme;
}

public String uneMemeChaine() {
    String chaine="";
    if (this.nom.equals("truc"))
    {chaine = "truc";
    }
    else
        if (this.nom.equals("truc2")) {
            chaine = "truc";
        } else {
            if (this.nom.equals("truc3")) {
                chaine = "truc";
            }
        }
}

```

```

        return chaine;
    }
    public String unAgeOuAutre() {
        String chaine = "pastruc282148";
        if ((this.age == 2)|| (this.age == 8)
            || (this.age == 21) || (this.age == 48))
        {
            chaine = "truc282148";
        }

        return chaine;
    }
    public String niveauEtu() {
        String niv = "";

        switch(niveau)
        {
            case 1:
                niv+="bac+1";
                break;
            case 2:
                niv+="bac+2";
                break;
            case 3:
                niv+="bac+3";
                break;
            case 4:
                niv+="bac+4";
                break;
            case 5:
                niv+="bac+5";
                break;
            default : niv += "Pas de Bac";
        }

        return niv;
    }
}

```

```

public String getAppreciation() {
    String app = "";
    if (classement1==classement2)
    {
        if (classement1 >= 10 )

        switch (niveau)
        {
            case 1:
                app="première année validée";
                break;
            case 2:
                app="année 2 validée";
                break;
            case 3:

```

```

        app="licence obtenue";
        break;
    case 4:
        app="première année de master valide";
        break;
    case 5:
        app="master obtenu";
        break;

    default:
        app = "valable";
        break;

    }
    else app += "non valide";
}
else if (classement1>=10 || classement2>=10)

{
    int minusResult = classement1-classement2;
    if (minusResult> 3 ) app ="des resultats en forte baisse au semestre 2";
    else if (minusResult > 1) app ="resultats en légère baisse au semestre 2";
    else if (minusResult< 3) app = "résultat en fort progres";
    else app = "résultat en progres";
    app+= " année valide";
}
else app += "non valide";

return app;
}

```

```

public String getResultAnnee(){ // faire des switch
    String res = " ";
    String pres1 ="";

    String pres2 ="";
    if (classement1 == classement2 && classement1 < 4)
    {
        switch (classement1) {
            case 0 : res = "Genial";
                break;
            case 1 : res = "tres bien";
                break;
            case 2 : res = "bien";
                break;
            default :
                }
            res += "- sem1";
        }
    if (classement1==classement2 && classement1>=3)
        res = "Fabuleux";
}

```

```

if (classement1 > 0 && classement2==0)
{
    switch (classement1) {
        case 1 : pres1 = "Moyen";
            break;
        case 2 : pres1 = "très bien";
            break;
        case 3 : pres1 = "bien";
            break;
        default :
    }
    pres2 = "Génial";
    res = pres1 + "-" + pres2;
}
if (classement2 > 0 && classement1==0)
{
    switch (classement2) {
        case 1 : pres2 = "moyen";
            break;
        case 2 : pres2 = "Tres bien";
            break;
        case 3 : pres2 = "bien";
            break;
        default :
    }
    pres1 = "Génial";
    res = pres1 + "-" + pres2;
}

if (classement1>classement2 && classement1 < 4)
{
    if (classement1==2)
        pres1="Tres bien";
    if (classement1==3)
        pres1="bien";
    if (classement2==1)
        pres2="moyen";
    if (classement2==2)
        pres2="bien";
    res = pres1 + "-" + pres2;
}
if (classement2>classement1 && classement2 < 4)
{
    if (classement2==2)
        pres2="bien";
    if (classement2==3)
        pres2="Moyen";
    if (classement1==1)
        pres1="insuffisant";
    if (classement1==2)
        pres1="bien";
}

```

```

        res = pres1 + "-" + pres2;
    }

    if (classement1 > classement2 && classement2 >= 3)
    {
        res = "Semestre 1 meilleur que semestre 2";
    }

    if (classement2 > classement1 && classement1 >= 3)
    {
        res = "Semestre 2 meilleur que semestre 1";
    }

    if (classement1>=4 && classement2>=0 && (classement1-classement2)>=2)
    {
        res = "très bon semestre 1 - Bravo";
    }
    if (classement2>=4 && classement1>=0 && (classement2-classement1)>=2)
    {
        res = "très bon semestre 2 - Bravo";
    }
    return res;
}

}

```

Chapitre 2 :

```

public class Compteur {

    // attributs privés
    private int kilometrage;
    private int vitesse;
    private int vitesseMax;

    // Constructeur
    public Compteur() {
        this.kilometrage = 0 ;
        this.vitesse = 0;
        this.vitesseMax = 130;
    }

    public void setKmParcours(int nbKms) {
        this.kilometrage = nbKms;
    }

    public int getVitesse() {

```



```

        return this.vitesse;
    }

    public void setVitesse(int vitesse) {
        this.vitesse = vitesse;
    }
}

public class Reservoir {

    // attributs privés
    private int capacite;
    private int contenu;
    private int prixLitreEssence;

    // Constructeur
    public Reservoir(int unePuissance) {
        if(unePuissance<=6) {
            this.capacite = 40;
        }
        else {
            this.capacite = 60;
        }
        this.contenu = this.capacite;
        this.prixLitreEssence = 1;
    }

    public float distanceMaxi (float conso) {
        float distance = 0;

        return distance;
    }

    public void diminuer(float conso, float nbKms) {

    }

    public void remplir() {

    }
}

public class Voiture {

    // attributs privés
    private int puissance;
    private String numImma;
    private Reservoir unReservoir;
    private Compteur unCompteur;

```

```

// Constructeur
public Voiture (int unePuissance, String unNumImma) {
    this.puissance = unePuissance;
    this.numImma = unNumImma;
    this.unReservoir = new Reservoir(unePuissance);
    this.unCompteur = new Compteur ();
}

private boolean tomberEnPanne() {
    if() {
        this.fairePlein();
    }
}

private int conso(int unePuissance, int vitesseInstantane) {
    int consommationEssence=0;
    if(vitesseInstantane> 0 & vitesseInstantane <= 80) {
        consommationEssence = 6;
    }
    else {
        if(vitesseInstantane> 80 & vitesseInstantane <= 100){
            consommationEssence = 7;
        }
    }
    {
        if (vitesseInstantane> 100 & vitesseInstantane <= 120) {
            consommationEssence = 8;
        }
    }
    {
        if (vitesseInstantane> 120 & vitesseInstantane <= 130) {
            consommationEssence = 9;
        }
    }
    consommationEssence *= (1+0.15*unePuissance);
    return consommationEssence;
}

private void fairePlein() {

}

public void rouler(int vitesseConstante, int nbKmsPrevu) {
    int depannage=0;
    if(this.tomberEnPanne()) {
        depannage=100;
        this.fairePlein();
        nbKmsPrevu = 0;
    }
}

```

```

        public void stoper() {
        }
    }
}

```

Convertisseur :

```
package convertisseur;
```

```

public class ConvertisseurBasique {

    //attributs privés
    private static double TAUX_CONVERSION;

    public ConvertisseurBasique() {

    }

    public double convertirEnEuros(double nombreEnYens) {
        double resultatEnEuros=0;
        resultatEnEuros = nombreEnYens/100;
        return resultatEnEuros;
    }

    public double convertirEnYens(double nombreEnEuros) {
        double resultatEnYens=0;
        resultatEnYens = nombreEnEuros*100;
        return resultatEnYens;
    }
}

```

```
package convertisseur;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
import org.junit.jupiter.api.Test;
```

```

class ConvertisseurBasiqueTest {

    @Test
    /*
    * void test() { fail("Not yet implemented"); }
    */

    void testConvertisseur() {
        ConvertisseurBasique conv = new ConvertisseurBasique();
        // test avec un nombre nul
        assertEquals(0.0, conv.convertirEnYens(0));
        assertEquals(0.0, conv.convertirEnEuros(0));
        // test avec le taux de conversion officiel
        assertEquals(1, conv.convertirEnYens(0.01));
    }
}

```

```

        assertEquals(1.0, conv.convertirEnEuros(100));
        // test avec un nombre quelconque (100 x 132.11 = 13211)
        assertEquals(100, conv.convertirEnEuros(10000));
        assertEquals(10000, conv.convertirEnYens(100));
    }
}

```

```
package convertisseur;
```

```

public class Operation {

    public Operation() {

    }

    public double additionner(double nombre1, double nombre2) {
        double resultat = 0;
        resultat = nombre1+nombre2;
        return resultat;
    }
}

```

```
package convertisseur;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
import org.junit.jupiter.api.Test;
```

```

class OperationTest {

    @Test
    /*
     * void test() { fail("Not yet implemented"); }
     */

    void testAdditionner() {

        Operation ope = new Operation();
        // test d'une addition
        assertEquals(9,ope.additionner(6,3));
        // test avec une autre méthode
        assertTrue(9==ope.additionner(6,3));
    }
}

```

EDC 0 :

```
import java.util.List;
```

```
public class Magasin {
```

```
    //attributs privés
```

```

//collection des pièces non agréées
private List<PièceNonAgréée>lesPiècesNonAgréées;

//constructeur
public Magasin( List<PièceNonAgréée> lesPièces) {
    this.lesPiècesNonAgréées = lesPièces;
}

//méthodes publiques

//L'état de la pièce passée en paramètre (de type entrée-sortie) prend la valeur "Rouge"
public void REBUTER (PièceNonAgréée laPièce) {
    laPièce.setRouge();
}

/**
 * @param nbHeures
 */
public void REVISER (int nbHeures) { // a tester
    for(PièceNonAgréée laPièce : this.lesPiècesNonAgréées){
        if(laPièce.getEtat().equals("Vert")&&(laPièce.getNbHeures())>=nbHeures)){
            laPièce.setOrange();
        }
    }
}

/**
 * @param num
 */
public void SUPPRIMER (String num) { // a tester
    for(PièceNonAgréée laPièce : this.lesPiècesNonAgréées){
        if(laPièce.getNumSerie().equals(num)){
            this.lesPiècesNonAgréées.remove(laPièce);
        }
    }
}

/**
 * @return le nombre de pièces
 */
public int getNbPieces() {
    return this.lesPiècesNonAgréées.size();
}

/**
 * @param nbPieceAjout et laPièce
 */
public void ajoutPiece(PièceNonAgréée laPièce ) { // après modification
    this.lesPiècesNonAgréées.add(laPièce);
}

```

```

@Override
public String toString() { // à tester
    String valider="";
    for(PièceNonAgréée laPièce : this.lesPiècesNonAgréées) {
        valider += laPièce.toString();
    }
    return valider;
}
}

```

```

abstract class Pièce {
    //attributs privés
    private String libelléPièce;
    private String numSerie;
    private int nbHeures;

    // constructeurs
    public Pièce(String libelléPièce, String numSerie, int nbHeures) {
        this.libelléPièce = libelléPièce;
        this.numSerie = numSerie;
        this.nbHeures = nbHeures;
    }

    //méthodes publiques

    abstract boolean methodeAbstraite();
    /**
     * @return le libellé de la pièce
     */
    public String getLibelléPièce() {
        return this.libelléPièce;
    }
    /**
     * @param libelléPièce
     */
    public void setLibelléPièce(String libelléPièce) {
        this.libelléPièce = libelléPièce;
    }

    /**
     * @return le numéro de série de la pièce
     */
    public String getNumSerie() {
        return this.numSerie;
    }
    /**
     * @param numSerie
     */
    public void setNumSerie(String numSerie) {
        this.numSerie = numSerie;
    }
}

```

```

/**
 * @return le nombre d'heures
 */
public int getNbHeures() {
    return this.nbHeures;
}
/**
 * @param nbHeures
 */
public void setNbHeures(int nbHeures) {
    this.nbHeures = nbHeures;
}

@Override
public String toString() {
    return this.libelléPièce + this.numSerie + this.nbHeures ;
}

}

import java.time.LocalDate;

public class PièceAgréée extends Pièce {
    //attributs privés
    private LocalDate dateAgrément;
    private String nomConstructeur;

    //constructeur
    public PièceAgréée(String libelléPièce, String numSerie, int nbHeures, LocalDate
dateAgrément,String nomConstructeur) {
        super(libelléPièce, numSerie, nbHeures);
        this.dateAgrément = dateAgrément;
        this.nomConstructeur = nomConstructeur;
    }

    //méthodes publiques
    public boolean methodeAbstraite() {
        return true;
    }
    /**
     * @return la date de l'agrément
     */
    public LocalDate getDateAgrément() {
        return this.dateAgrément;
    }
    /**
     * @param dateAgrément
     */
    public void setDateAgrément(LocalDate dateAgrément) {
        this.dateAgrément = dateAgrément;
    }
}

```

```

/**
 * @return le nom du constructeur
 */
public String getNomConstructeur() {
    return this.nomConstructeur;
}
/**
 * @param nomConstructeur
 */
public void setNomConstructeur(String nomConstructeur) {
    this.nomConstructeur = nomConstructeur;
}

@Override
public String toString() {
    return super.toString()+this.dateAgrément+ this.nomConstructeur;
}
}

public class PièceNonAgréée extends Pièce {
    //attributs privés
    private String etat;

    //constructeur
    public PièceNonAgréée(String libelléPièce, String numSerie, int nbHeures, String etat) {
        super(libelléPièce, numSerie, nbHeures);
        this.etat = etat;
    }

    //méthodes publiques
    public boolean methodeAbstraite() {
        return true;
    }
    /**
     * @return l'état de la pièce non agréée
     */
    public String getEtat() {
        return this.etat;
    }

    /**
     * modifie l'état de la pièce non agréée en "Rouge"
     */
    public void setRouge() {
        this.etat="Rouge";
    }
    /**
     * modifie l'état de la pièce non agréée en "Vert"
     */
    public void setVert() {
        this.etat="Vert";
    }
}

```



```

/**
 * modifie l'état de la pièce non agréée en "Orange"
 */
public void setOrange() {
    this.etat="Orange";
}

@Override
public String toString() {
    return super.toString()+ this.etat;
}

}

import java.util.ArrayList;

public class Test {
    public static void main (String args[]){

        // Création de la liste
        ArrayList<Pièce>lesPièces;
        ArrayList<PièceNonAgréée>lesPiècesNonAgréées;

        // declaration d'un objet de la classe Pièce
        Pièce p1, p2, p3;

        // declaration d'un objet de la classe PièceNonAgréée
        PièceNonAgréée p4, p5, p6;

        Magasin m1;

        int tailleListe;

        // instantiation des objets de la classe
        p1 = new Pièce ("Anémomètre ", "HJDFKDFS ", 10);
        p2 = new Pièce ("Horizon artificiel ", "KJDLKMKJ", 12);
        p3 = new Pièce ("Boîte noire ", "BNQSLKEE", 24);
        p4 = new PièceNonAgréée("Hublot côté gauche ", "IUTRHJGF ", 24, " En attente");
        p5 = new PièceNonAgréée("Hublot côté droit ", "DGFSJKLM ", 24, " En attente");
        p6 = new PièceNonAgréée("Hublot central ", "DFSHGIUO ", 24, " En attente");

        lesPièces = new ArrayList<Pièce>(); // instantiation de la collection
        lesPiècesNonAgréées = new ArrayList<PièceNonAgréée>();

        m1 = new Magasin(lesPiècesNonAgréées);
        // Ajout des objets dans la collection Pièce
        lesPièces.add(p1);
        lesPièces.add(p2);
        lesPièces.add(p3);
    }
}

```

```

// Ajout des objets dans la collection PièceNonAgréée
lesPiècesNonAgréées.add(p4);
lesPiècesNonAgréées.add(p5);
lesPiècesNonAgréées.add(p6);

System.out.println("Voici la liste des pièces détachées dans notre stock :");
for(Pièce p : lesPièces) {
    System.out.println(p);
}

// Suppression de l'objet à l'indice 2 dans la collection
lesPièces.remove(2);

System.out.println("Voici la nouvelle liste des pièces détachées dans notre stock :");
for(Pièce p : lesPièces) {
    System.out.println(p);
}

System.out.println("-----");

System.out.println("Voici l'état des pièces non agréées avant modification : ");
for(PièceNonAgréée p : lesPiècesNonAgréées) {
    System.out.println(p);
}

System.out.println("Voici l'état des pièces non agréées après modification : ");
p4.setVert();
p5.setOrange();
p6.setRouge();
for(PièceNonAgréée p : lesPiècesNonAgréées) {
    System.out.println(p);
}

System.out.println("-----");

System.out.println("Le nombre de pièces dans le magasin est ");
System.out.println(m1.getNbPieces());
System.out.print(m1.toString());
}
}

```

Logiciel :

```

package produitscdes;
import java.util.HashMap;
import java.util.Map;

public class Commande {
    // attribut numCde
    private int numCde;
    // le type primitif int n'est pas possible dans HashMap ==> on utilise un attribut de la classe
    Integer

```

```

private HashMap <Produit,Integer> produitscommande = new HashMap<>();

public Commande(int numCde){
    this.numCde = numCde;
}

public void ajouterProduit(int idProduit,String nom, double prix, Integer quantite) throws
Exception {

    if(quantite>0) {
        if (!(this.produitscommande.containsKey(idProduit))){ //On recherche
idProduit dans la HashMap

        }
        else throw new Exception("Le produit " + nom + " est déjà présent"); // On ne
peut pas ajouter à la commande un produit déjà existant.
    }
}

public void ajouterProduit(Produit p, Integer quantite) throws Exception {
    if(quantite>0) {
        this.produitscommande.put(p,quantite);
    }
}

public double totalCommande() {
    double total = 0;

    for(Map.Entry<Produit,Integer> p : produitscommande.entrySet()) {
        total += p.getKey().getPrix()*this.produitscommande.get(p.getKey());
    }
    // le total est égal à la somme (quantité commandé * prix)

    return total;
}
}

```

```

package produitscdes;

```

```

import static org.junit.jupiter.api.Assertions.*;

```

```

import java.util.HashMap;

```

```

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

```

```

class CommandeTest {

```

```

    Commande c0;

```

```
Commande c1;  
Commande c2;
```

```
Produit p1;  
Produit p2;  
Produit p3;
```

```
@BeforeAll  
static void setUpBeforeClass() throws Exception {  
  
}
```

```
@BeforeEach  
void setUp0() throws Exception {  
    c0 = new Commande(0);  
}
```

```
@Test  
void testTotalCommande0prod() throws Exception {  
    try {  
        assertEquals(0,c0.totalCommande());  
    }  
    catch(Exception e) {  
        fail("Erreur total commande pour 0 produit");  
    }  
}
```

```
@AfterEach  
void tearDown0() throws Exception {  
}
```

```
@BeforeEach  
void setUp1() throws Exception {  
    p1 = new Produit(1,"Tartiflette",10);  
    c1 = new Commande(1);  
  
}
```

```
@Test  
void testTotalCommande1prod() throws Exception {  
    try {  
        c1.ajouterProduit(p1,5);  
        assertEquals(50,c1.totalCommande());  
    }  
    catch(Exception e) {  
        fail("Erreur total commande pour 1 produit");  
    }  
}
```

```
@AfterEach  
void tearDown1() throws Exception {  
}
```

```

@BeforeEach
void setUp2() throws Exception {
    p2 = new Produit(2,"Cassoulet",4);
    p3 = new Produit(3,"Raclette",6);
    c2 = new Commande(2);
}

@Test
void testTotalCommande2prod() throws Exception {
    try {
        c2.ajouterProduit(p2,12);
        c2.ajouterProduit(p3,5);
        assertEquals(78,c2.totalCommande());
    }
    catch(Exception e) {
        fail("Erreur total commande pour 2 produits");
    }
}

```

```

@AfterEach
void tearDown2() throws Exception {
}

```

```

@AfterAll
static void tearDownAfterClass() throws Exception {
}

```

```

}

```

```

package produitscdes;

```

```

public class Produit {
    // Attributs privés
    private int numProduit;
    private double prix;
    private String nom;

    //A compléter
    public Produit(int numProduit, String nom, double prix) throws Exception
    {
        this.numProduit = numProduit;
        this.nom = nom;
        if(prix>0)
        {
            this.prix = prix;
        }
        else
            throw new Exception("Le prix" + prix + " est incorrect !");
    }

    public int getNumProduit() {
        return this.numProduit;
    }
}

```

```

    }

    public void setNumProduit(int numProduit) {
        this.numProduit = numProduit;
    }

    public double getPrix() {
        return this.prix;
    }

    public void setPrix(double prix) {
        this.prix = prix;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getNom()
    {
        return this.nom;
    }

}

package produitscdes;
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class ProduitTest {

    @Test
    void testProduit() {
        int numProduit = 1;
        String nom = "huile essentielle lavande";
        double prix = 12.5;{
            if(prix>0) {
                try
                {
                    Produit p1 = new Produit(numProduit,nom, prix);
                    assertEquals(nom, p1.getNom());
                    assertEquals(prix, p1.getPrix());
                    assertEquals(numProduit, p1.getNumProduit());
                }
                catch (Exception e)
                { // erreur attendue
                }
            }
            else {
                fail("Erreur de prix");
            }
        }
    }
}

```

```
}
```

```
}
```

```
@Test  
public void testReglesMetier() throws Exception  
{  
    Produit p1;  
    try  
    {  
        p1 = new Produit(1,"sel de mer", 0);  
        fail("Erreur non détectée : zero incorrect");  
    }  
    catch (Exception e)  
    { // erreur attendue  
    }  
}
```

```
@Test  
public void testRemonteExceptionPrix0() {  
    assertThrows( RuntimeException.class, () -> {  
        new Produit(1,"sel de mer", 0);  
    });  
}
```

```
}
```